Foundations

System Design · Part 1

2026-10-22

Before designing large systems you need three foundations:

1. **How one computer works inside.** Disk, RAM, CPU and cache, and how data moves between them.
2. **How a real production app is put together.** Servers, storage, load balancers, logs, metrics and alerts.
3. **What makes a design good.** Availability, reliability, throughput and latency.

Summarised from the System Design course at neetcode.io.

How to revise: read each chapter's **At a glance** box. If you can explain every line out loud, you know the chapter. If a line feels fuzzy, read that section.

Contents

1 Computer Architecture 3
1.1 Units 3
1.2 Disk 4
1.3 RAM 4
1.4 CPU 4
1.5 Cache 5
1.6 Moore's Law 6
1.7 Which memory, when? 6
2 Application Architecture 7
2.1 The developer's view 8
2.2 The user's view 8
2.3 Scaling: vertical vs horizontal 8
2.4 Load balancer 9
2.5 External APIs 9
2.6 Logging and metrics 9
2.7 Alerts 9
2.8 What comes next 10

3 Design Requirements 10
3.1 Thinking in system design 10
3.2 Availability 11
3.3 Reliability, fault tolerance, redundancy 12
3.4 Throughput 13
3.5 Latency 13
3.6 Summary: the goal 14

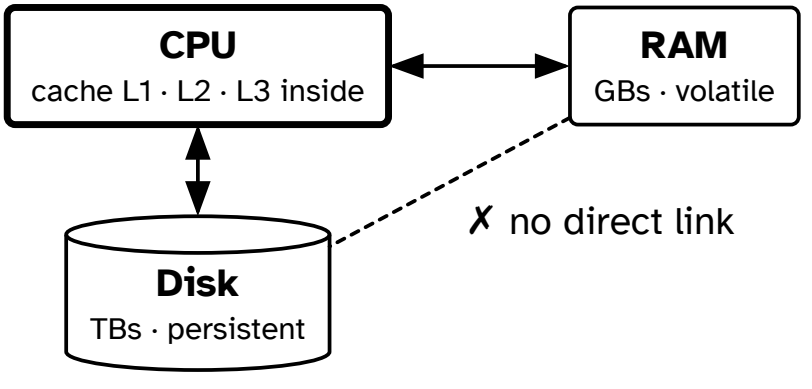
CHAPTER 1

Computer Architecture

AT A GLANCE

- **Four parts: Disk** is big, slow and permanent. **RAM** is smaller, fast and temporary. **CPU** does all the work and moves the data. **Cache** is tiny, the fastest, and sits inside the CPU.
- **Speed:** Cache > RAM > SSD > HDD. **Size:** the reverse order. **Cost per GB:** the same order as speed.
- RAM ≈ **microseconds**, disk ≈ **milliseconds**, so RAM is roughly **1000× faster**.
- **RAM and disk never talk directly.** The CPU sits between them.
- **Read order:** cache → RAM → disk. Use the fastest place that has the data.
- **Caching** means keeping a copy of data in a faster place. It's used everywhere: CPU, browser, servers.
- **Moore's law:** transistors double every ~2 years. That growth is now slowing.

↳ **Remember:** *Disk is the library (holds everything, keeps it forever, slow to fetch from). RAM is your desk (what you are working on now, cleared when you leave). Cache is the page in your hand. The CPU is you.*



How the parts connect: everything goes through the CPU

1.1 Units

A **bit** is the smallest unit: a single 0 or 1. A **byte** is 8 bits.

Unit	Bytes	Typical example
KB	10^3	CPU L1 cache
MB	10^6	L3 cache (tens of MB)
GB	10^9	RAM, USB drive
TB	10^{12} (a trillion)	Laptop disk

1.2 Disk

The **primary storage** device. It is **persistent** (non-volatile): the data stays when the machine is off. Typical sizes are **TBs**.

	HDD (hard disk drive)	SSD (solid-state drive)
How	Mechanical: spinning platter + moving read/write head	Electronic flash memory, **no moving parts**
Speed	Slower	**Much faster**
Ageing	Wears out, gets slower over time	No mechanical wear
Cost	Cheaper per GB	Costs more per GB

1.3 RAM

Random Access Memory holds the apps you have open and your program's **variables**.

- **Volatile:** it is erased on shutdown. That is why you save work to disk.
- **Size:** 1–128 GB, much smaller than disk because RAM is more **expensive** per GB.
- **Speed:** writing 1 MB takes about **µs** (10^{-6} s) in RAM and about **ms** (10^{-3} s) on disk. These are rough numbers and change as hardware improves.

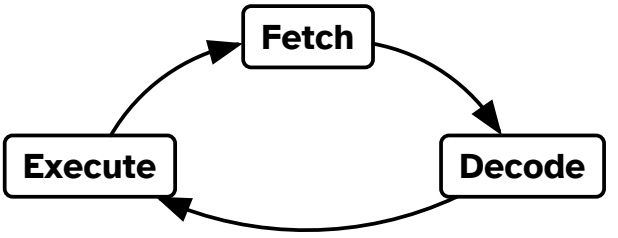
KEY RAM and disk do **not** communicate directly. The CPU moves data between them.

1.4 CPU

The **central processing unit** is the "brain". It **reads from and writes to RAM and disk**, and does **all computation** (add, subtract, multiply, compare...).

How your code runs:

1. Your code is translated into **binary instructions**, which are stored in **RAM**.
2. The CPU **fetches** an instruction from RAM, **decodes** it and **executes** it, then repeats.
3. Executing may touch data in RAM or on disk. Opening a file and reading it line by line is a read from disk.



At the lowest level, every instruction is just bytes. One instruction takes **under a nanosecond** (10^{-9} s), because CPUs run at GHz, which is billions of cycles per second.

1.5 Cache

Extremely fast memory on the same chip (die) as the CPU.

- Most CPUs have **L1, L2 and L3** caches. **L1** is the smallest and fastest, **L3** the largest. Sizes range from **KBs to tens of MBs**.
- **Why it is so fast:** it sits on the CPU die, **and** it is made of **SRAM**, which is faster than the **DRAM** used for main RAM.
- **Read order:** check the cache first, then RAM, then disk.
  - **Cache hit:** the data is in the cache and unchanged, so it is served from the cache.
  - **Cache miss:** go down a level to RAM, or to disk.
- **Who decides what is cached:** the **CPU hardware** manages L1, L2 and L3 automatically. The OS manages a different cache, the **page cache**, which keeps disk data in RAM.

KEY A cache is any **smaller, faster** store that keeps **copies** of data from a **slower** one.

Caching beyond the CPU: a web browser caches HTML, CSS, JS and images **on disk**. Reading from disk is slow compared with RAM, but it is still much faster than a network request. If the cached copy is still valid, the page loads faster.

1.6 Moore's Law

An **observation**, not a law of physics: the **number of transistors on a chip doubles about every 2 years**.

GROWTH

$$N(t) = N_0 \cdot 2^{t/2} \quad (t \text{ in years})$$

- On the usual chart it **looks linear**, but the **y-axis is logarithmic**, so the growth is **exponential**.
- As transistor counts doubled, the **cost of computing power halved**.
- In recent years the doubling has **slowed down and plateaued** because of physical limits. This is one reason large systems **add more machines** instead of waiting for faster ones.

1.7 Which memory, when?

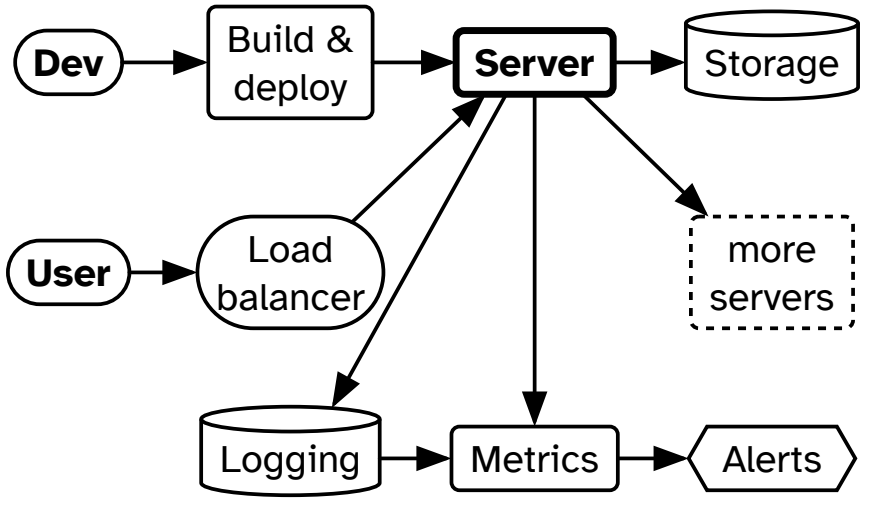
	Cache	**RAM**	**Disk**
Size	KB – tens of MB	GB	TB
Speed	Fastest	µs	ms
Survives power off?	No	No	**Yes**
Cost per GB	Highest	High	Low
Use it for	Hottest data the CPU needs right now	Open programs, variables (multitasking)	Anything that must survive, like your essay

Application Architecture

AT A GLANCE

- **Developer side:** write code → **build & deploy** → **server**. A server is a computer that **handles requests from other computers**.
- The server keeps its data in **external storage** (a database or cloud storage) over the **network**, because built-in storage is limited.
- **User side:** the user's **browser is the client**. The server replies with HTML, CSS and JS.
- **Too many users** → a **bottleneck** (CPU or RAM) → **scale**:
  - **Vertical** means a bigger machine. It's easy, but there's a hard ceiling.
  - **Horizontal** means more machines. It's powerful, survives a failure and uses cheap hardware, but needs more engineering.
- A **load balancer** spreads requests evenly across the servers.
- Servers also call **external services through APIs**. For example, neetcode.io calls Stripe for payments.
- **Observability: logs** record what happened, **metrics** show how resources are doing, **alerts** push a notification when a metric crosses a threshold.

↳ **Remember:** *Logs tell the story, metrics take the pulse, and alerts ring the bell.*



A production app: the developer path (top) and the user path (middle)

2.1 The developer's view

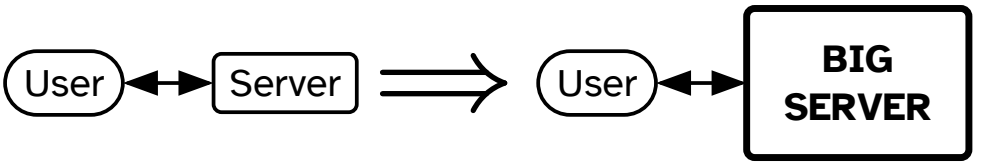
- Developers write code that is **deployed to a server**.
- **Server:** a computer that handles requests from another computer.
- The app needs **persistent storage** for its data. A server's built-in storage is **limited in size**, so it usually talks to **external storage** (a database or the cloud) **over a network**, not on the same machine.

2.2 The user's view

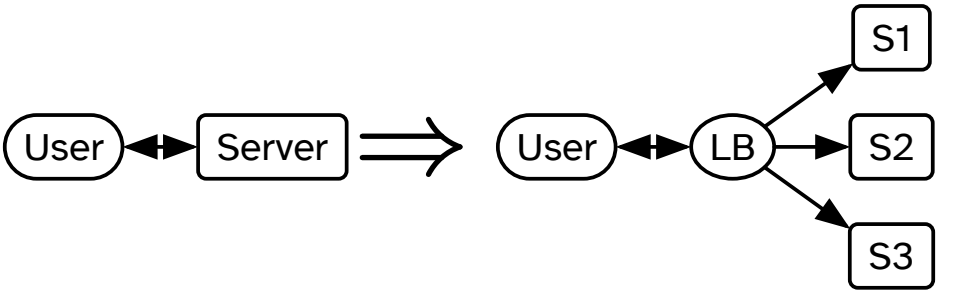
- A **user** sends requests to the server, usually through a **web browser**. The browser is the **client**.
- For a front-end feature, the server responds with the **HTML, CSS and JavaScript** needed to show it.
- With many users, one server becomes a **bottleneck** (its **CPU or RAM**). To keep it fast, we **scale**.

2.3 Scaling: vertical vs horizontal

	Vertical (scale up)	**Horizontal (scale out)**
What	Upgrade the same machine: more RAM, more CPU cores, higher clock speed	Add **more servers** running the same code and split the requests between them
Limit	Every machine has a **max upgrade**	Keep adding machines
One server dies?	**Everything is down**	Traffic moves to the **other servers**
Hardware	Expensive high-end machines	Cheap **commodity** hardware
Effort	**Easy**	**Hard**: servers must coordinate, and requests must be spread evenly
Use when	Simple apps	Large systems (the usual choice)



Vertical: a bigger box. 100 → 150 requests/second



Horizontal: more boxes. 100 → 300 requests/second

Example: Amazon Prime Video moved some services from microservices back to a **monolith**. Simpler designs are sometimes the better choice.

2.4 Load balancer

With many servers, the **load balancer** decides **which server gets each request**. It **spreads incoming requests evenly** across the group.

2.5 External APIs

Servers rarely work alone. They call **other services through APIs**. For example, **neetcode.io** uses **Stripe** for payments.

2.6 Logging and metrics

- **Logs** record **all the activity**: how requests went, any **errors**, and what happened **before a crash**. Logs can live on the same server, but writing them to an **external server** is **more reliable**.
- Logs don't show the full picture. You can't see **RAM or CPU becoming the bottleneck** in them.
- A **metrics service** collects numbers from across the server environment, such as **CPU usage, RAM usage and network traffic**, so you can spot **bottlenecks**.

2.7 Alerts

Checking metrics by hand is like checking your phone every 5 minutes for messages. Instead, set **alerts**: when a metric **misses its target**, developers get a **push notification**.

Example: The target is 100% of requests succeeding. Alert if the success rate drops **below 95%**.

2.8 What comes next

All these parts may sit on **different computers**, so they need **networking**: how they communicate, which **protocols** they follow, and how to optimise those protocols.

CHAPTER 3

Design Requirements

AT A GLANCE

- **Interviews** test your **thought process and trade-offs**, not every detail.
- Every big system does three things: **Move data · Store data · Transform data**.
- **Bad code is cheap to fix. Bad design is expensive** (for example, migrating to a new database and rewriting the app).
- A design is **good** when it scores well on **A Real Tough Load**:
  - **Availability:** the % of time it is up. Measured in **nines**. 99.999% ≈ **5 min** down per year.
  - **Reliability:** the chance it doesn't fail. It needs **fault tolerance**, which needs **redundancy**.
  - **Throughput: how many** per second (req/s, QPS, bytes/s).
  - **Latency: how long** one request takes.
- **SLA** is the promise to customers. **SLO** is the internal target that keeps the promise.

↳ **Remember:** *Move · Store · Transform is what a system does. A Real Tough Load is how you judge it.*

3.1 Thinking in system design

Task	What it means and the question to ask
Move data	Data moves between **clients and servers spread across the world**. This is much harder than moving it between disk, RAM and CPU inside one machine.

Task	What it means and the question to ask
Store data	**Databases? Blob stores? File systems? Distributed file systems?** Choosing one is like choosing an **array or a BST**: neither is always better. **It depends on the use case.**
Transform data	Turn raw data into something useful. Examples: server logs → **% successful vs % failed requests** (often done by a **monitoring service**), or medical records → **patients filtered by age**. Ask: what is the **most efficient** way?

KEY Bad **code** is cheap to fix: rewrite it. A bad **architecture** is costly. The wrong database means **migrating the data** and **rewriting parts of the app**.

3.2 Availability

The % of time the system is up and running over a given period. Systems used to need 9am–5pm on weekdays. Now they need to be up **24/7, worldwide**, with requests arriving from every timezone at once.

AVAILABILITY

$$\text{availability} = \frac{\text{uptime}}{\text{uptime} + \text{downtime}}$$

- **Planned downtime:** software updates, maintenance, backups.
- **Unplanned downtime:** hardware or software failure, natural disaster. You can't predict it, but you can **make it less likely**.

Example: Up for 23 of 24 hours: $23/24 \approx 96\%$. That sounds high, but it is **poor**. You lose users and money for an hour every day.

100% is impossible, so companies aim for **at least 99%** and measure availability in **nines**:

Availability	Downtime / year	Note
99% ("two nines")	3.65 **days**	Minimum target
99.9%	8.76 **hours**	10× less downtime
99.99%	52.6 **minutes**	AWS monthly SLA

Availability	Downtime / year	Note
99.999% (“five nines”)	5.26 **minutes**	Mission-critical systems

↳ **Remember:** *Each extra 9 means 10× less downtime: days → hours → under an hour → about 5 minutes.*

SLA and SLO

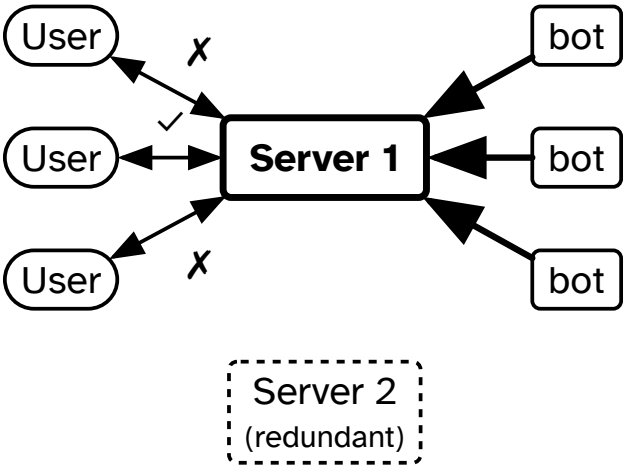
- **SLA (service level agreement):** the **promise to clients or users** about uptime, responsiveness and responsibilities. AWS promises 99.99% a month and **refunds service credits** if it misses.
- **SLO (service level objective):** the **internal target** your team must hit so the SLA is met.

3.3 Reliability, fault tolerance, redundancy

These three build on each other: **redundancy** gives you **fault tolerance**, and fault tolerance gives you **reliability**.

- **Reliability** is the **probability the system performs without failing** over a period of time. Availability asks “is it up right now?”. Reliability asks “**how easily does it go down?**”, for example under thousands of users or a DDoS attack.
- **Fault tolerance** is how well the system **detects and heals itself** when a part fails. It might disable a feature, switch to a different mode, or **switch to another server**.
- **Redundancy** means having **extra components you don’t normally need**, such as a backup server that **shadows** the primary and takes over if it fails. **Without redundancy there is no fault tolerance.**

Type	How it works
Active-passive	The backup **sits idle** and steps in only when the primary fails
Active-active	**Both servers handle traffic** all the time. This is what “redundancy” usually means.



A DDoS attack: bots flood Server 1, so most real users get no response

⚠ **CAREFUL** Redundancy is **not a perfect fix**. In a DDoS attack, Server 2 takes over, but the flood overloads it too.

3.4 Throughput

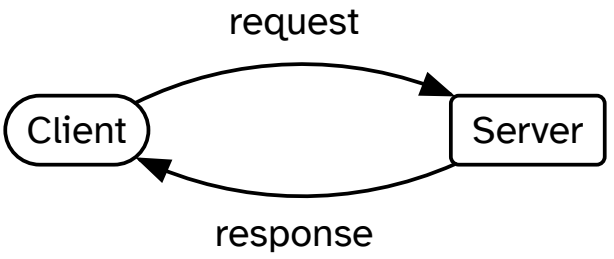
The amount of work handled per unit of time.

Measure	Use it when
Requests/second	Clients making requests to a server
Queries/second (QPS)	Requests to a server or database. It suits **user-facing** systems.
Bytes/second	**Data pipelines** processing large data, where there is no single user to count

- **To increase throughput:** scale **vertically** first, but it has limits. Then scale **horizontally**. The figures are in section 2.3.
- **Throughput vs bandwidth:** bandwidth is the **maximum** data rate. Throughput is what you **actually** get, which is always ≤ the bandwidth.

3.5 Latency

The delay between a client sending a request and getting the response: the time **one** request takes.



Round-trip latency

- **Throughput** is **how many** requests per second. **Latency** is **how long** each one takes.
- Latency exists **inside a computer too**: the CPU reading from cache, RAM or disk (see chapter 1).

↳ **Remember:** *Think of a highway. Throughput is cars per hour. Latency is how long one car's trip takes. Adding lanes raises throughput but doesn't shorten the drive.*

3.6 Summary: the goal

Design systems that **handle failures** (reliability), have **high availability** and **high throughput**, and **low latency**. Later chapters cover what affects each of these.